sized as power-of-two multiples of baseline pages. For example, x86 systems use 4KB baseline pages and support 2MB and 1GB superpages. Furthermore, superpages must be aligned in both virtual and physical memory (superpages of size N must begin at virtual and physical addresses that are multiples of N).

While superpages lower TLB miss rates by replacing hundreds to thousands of base page translations with a single superpage translation entry, they have management overheads [23, 28]. For example, dedicated OS code is required to support multiple large page sizes [25, 28, 29]. The process of ensuring that sufficient contiguous physical pages are allocated to virtual pages can suffer high performance overheads, particularly when alignment restrictions are also imposed [15, 23]. Furthermore, if not fully-utilized, superpages can increase the amount of I/O traffic and increase page initialization/fault latency. A particular problem is the use of a single dirty bit for all the baseline pages of a superpage; if set, the entire superpage must be written back to disk even if only one base page has been modified, greatly increasing disk traffic. Therefore, the OS weighs these overheads against the benefits of superpaging. Typically, the OS uses superpages sparingly, only bothering to generate large amounts of contiguity when superpaging is deemed to be worthwhile [4].

2.3. TLB Subblocking and Speculation

Two hardware schemes have been proposed to mitigate superpaging overheads. Talluri and Hill [28] present complete-subblock and partial-subblock TLBs, which record ranges of physical pages per virtual page entry. Complete subblocking, while effective, requires non-trivial modifications to traditional TLB hardware. Partial-subblocking overcomes these overheads but with explicit OS support [28]. Furthermore, subblocking was originally proposed for fully-associative designs rather than the set-associative organizations most commonly used in products today. Finally, sub-blocking effectiveness depends heavily upon page alignment. Both complete and partial subblocking are most effective when the first virtual page of a contiguous group of virtual (and physical pages) is aligned to the subblock length. Partial subblocking goes beyond this, reducing hardware overheads by requiring that base physical pages also be placed in an aligned manner within subblock regions [28]. Overall, these requirements may constrain exploitable contiguity.

Alternately, past work [6] proposes TLB speculation for systems with reservation-based superpaging [23, 28]. Here, physical pages are allocated in aligned 2MB regions of physical memory corresponding to their alignment within a 2MB region of virtual memory. Barr, Cox, and Rixner exploit this property with a SpecTLB structure, which interpolates between existing TLB entries to predict physical translations for TLB misses. While effective, SpecTLB requires reservation-based superpaging, which is not universally used (eg. Linux superpaging [4] does not use reservation-based superpages). SpecTLB also requires an additional TLB-like structure, and can increase instruction replays on incorrect speculations.

2.4. Our Approach

In general, there are three regimes of page contiguity that the OS generates. In the first regime, the OS does not succeed in mapping contiguous physical pages to contiguous virtual pages. In these cases, traditional techniques such as changing TLB organizations or prefetching are likely to be successful in increasing hit rates [6, 11, 19]. At the other end of the spectrum, there exists a regime of extremely high contiguity, when the OS decides that the overheads of superpage construction are well worth the effort. Our goal, in this work, is to exploit a third regime with intermediate contiguity, where tens to hundreds of base pages are contiguous. Our characterization studies will show that the OS naturally produces this contiguity without the overheads of superpages and that this level of contiguity is more prevalent than the other regimes. Our studies will be on a real system under a comprehensive set of system environments which include heavily-fragmented systems and systems with and without superpaging.

We will then realize low-overhead TLB hardware to exploit intermediate contiguity. Unlike prior work on speculation or prefetching [6, 11, 19], CoLT does not augment the standard TLBs with separate structures. Unlike superpages or subblocking, we avoid OS intrusion and do not require prescribed amounts of contiguity. Instead, CoLT studies available contiguity and exploits as much of it as possible.

3. Understanding Page Allocation Contiguity

We now explore why operating systems often allocate contiguous physical page frames to contiguous virtual pages. Since CoLT relies on this behavior, we ascertain which memory allocation policies and mechanisms produce contiguity.

3.1. Defining Page Allocation Contiguity

We say that system contiguity exists when consecutive virtual pages are allocated consecutive physical page frames. For example, if virtual pages *1*, *2*, and *3* are allocated physical page frames *58*, *59*, and *60*, we say that these pages are contiguous. Moreover, since this example involves three pages, we say that this is an instance of *3-page* contiguity.

Our definition is distinct from superpages in two ways. First, superpages require a set amount of contiguity. For example, 2MB superpages on x86 systems require instances of *512-page* contiguity. Instead, we make no restrictions on the amount of contiguity that is useful. Second, unlike superpages, we make no assumption on alignment. Our relaxations on contiguity amounts and alignment restrictions reveal huge great intermediate contiguity.

Note that this definition requires simultaneous contiguity in both virtual *and* physical page numbers. As such, CoLT does not affect cases where only virtual (or physical) pages are contiguous; these will be cached by TLBs in the conventional manner.

3.2. Sources of Page Allocation Contiguity

Operating systems maintain a complex set of policies and mechanisms to determine how to allocate physical memory to applications. A number of these policies promote physical page contiguity. We elaborate on them here, focusing on Linux for our discussion. Note, however, that our observations extend more broadly to other operating systems which tend to utilize similar mechanisms to allocate physical pages.

3.2.1. Buddy allocation. Linux uses a buddy allocator to track physical pages and assign them to virtual pages on demand. Figure 1 illustrates the operation of a buddy allocator, assuming that pages 1, 2, and 3 are already allocated. All free physical pages or page frames (PFs) are grouped into ten lists of blocks, which we refer to as *free lists*. Entry x in the free list tracks groups of 2^x *contiguous* physical pages. For example, pages 4-7 have 4-page contiguity and are hence listed by entry two.

Physical page allocations proceed as follows. Suppose an application requires an N-page data structure. To accommodate this, the